

MySQL Project: **Retail Analytics**

An End-to-End SQL Project from Database Design to Advanced Analytics

3,000

REGISTERED CUSTOMERS

600

CATALOG PRODUCTS

10,000

RECORDED ORDERS

21,932

BASKET LINE ITEMS

| Introduction



Profile

My name is **Bhanu Pratap Singh**, and I am an aspiring Data Analyst with a strong interest in SQL, database management, and data-driven decision making.

Through continuous learning and hands-on projects, I have developed skills in data analysis, querying, and database design.



Project Overview

This project, **Retail Analytics**, was developed using MySQL to analyze registered customers, catalog products, recorded orders, and basket line items.

The project demonstrates the practical application of SQL concepts—including table creation, constraints, joins, aggregate functions, subqueries, and window functions—to generate meaningful business insights.

| Introduction



Project Objective

- To design and manage an Online Movie Streaming Platform database using MySQL.
- To store and analyze user subscriptions, movie details, ratings, and watch history.



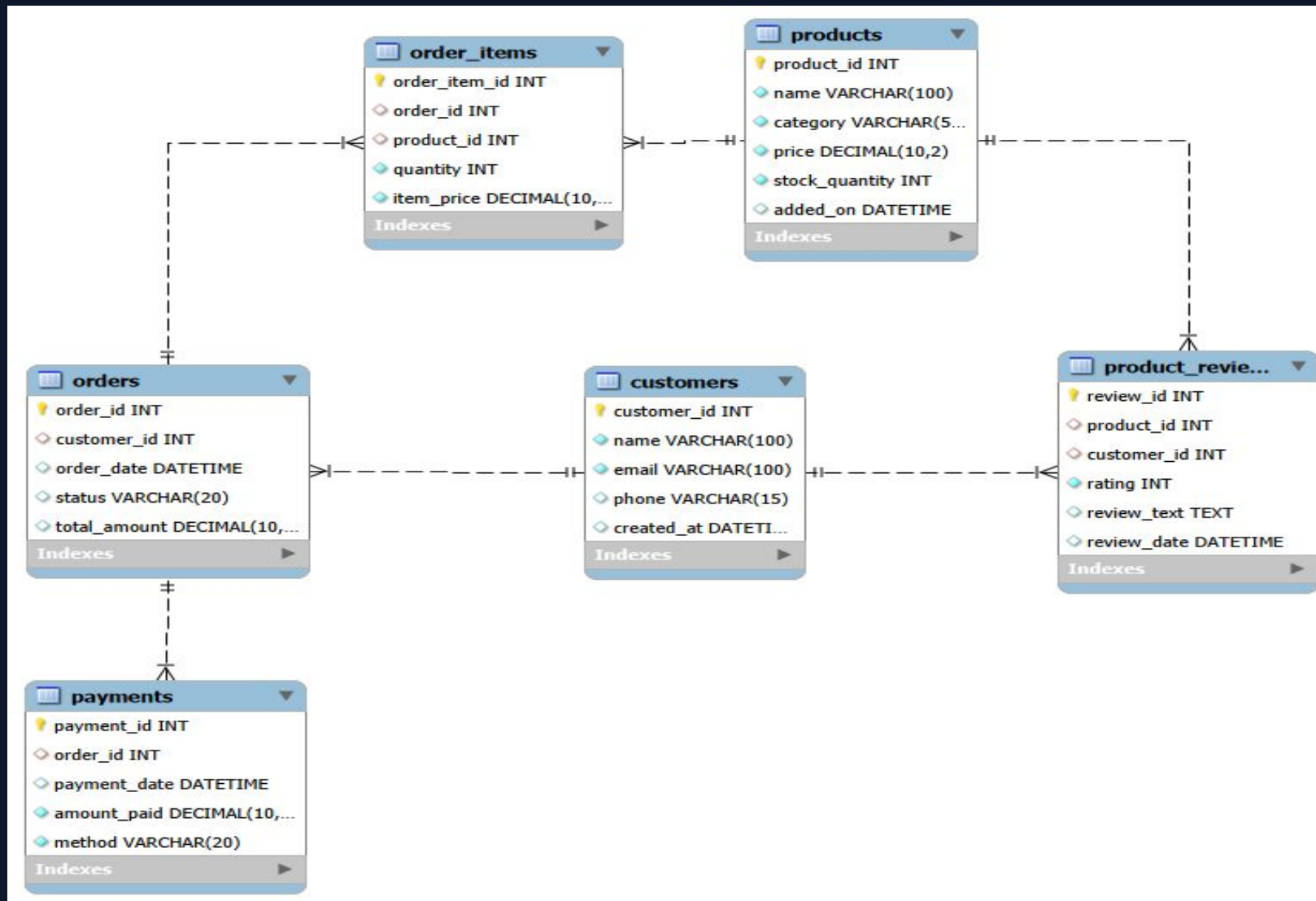
Project Goals

- Apply SQL concepts such as joins, aggregate functions, subqueries, and window functions.
- Generate business insights from streaming platform data.
- Improve database design and analytical skills.

Database Schema

Table Name	Key Columns	Primary Key	Foreign Keys	Function
customers	name, email, phone, created_at	customer_id	None	Stores 3,000 unique registered customer profiles.
products	name, category, price, stock_quantity	product_id	None	Catalog of 600 merchandise items across 10 categories.
orders	order_date, status, total_amount	order_id	customer_id	Transaction header storing status and overall order total.
order_items	quantity, item_price	order_item_id	order_idproduct_id	Bridge table splitting 21,932 basket items per order.
payments	payment_date, amount_paid, method	payment_id	order_id	Financial settlement transactions (UPI, Credit, NetBanking).
product_reviews	rating, review_text, review_date	review_id	product_idcustomer_id	Customer feedback ledger with 1 to 5 star ratings.

Enhanced Entity-Relationship (EER) Diagram





ANALYSIS I

ANALYSIS II

SALES

CY 733.22K | PY 609.21K

▲ 20.36%



PROFIT

CY 93.44K | PY 81.80K

▲ 14.24%



ORDERS

CY 3312 | PY 2587

▲ 28.02%



RETURNS

CY 289 | PY 197

▲ 46.70%



SALES TREND



BUSINESS OVERVIEW BY REGIONS

Region	Total Sales	Total Orders	Total Returns	Total Profit	Profit Ratio
West	720,123.45	2848	149	91,522.78	1.16
East	678,781.24	2848	149	91,522.78	1.16
South	501,239.89	2323	92	39,706.36	1.09
Central	501,239.89	2323	92	39,706.36	1.09

RANKING OVERVIEW

Value: ☐ Top ☐ Bottom

Metric: ☐ Total Sales ☐ People ☐ State ☐ City ☐ Customer

Measure By: ☐ Sales ☐ Profit ☐ Orders ☐ Returns

Ranking Option:

Adjust the slider as per requirements

WHICH SEGMENT CONTRIBUTED MOST TOWARDS TOTAL SALES?

Sum of Sales by Segment, Category and Sub-Category



SALES GROWTH ANALYSIS FOR TOP 5 SUB CATEGORIES



Ranking	State	Sales
457688	California	457,687.63
310876	New York	310,876.27
138641	Washington	138,641.27
76270	Michigan	76,269.61
70637	Virginia	70,636.72

Reset Filters



Realized Revenue, Total Orders & Average Order Value (AOV)

Evaluates primary store financial performance, filtering out cancelled orders to compute true realized revenue.

</> SQL Query

```
300 • SELECT
301     COUNT(order_id) AS valid_orders,
302     ROUND(SUM(total_amount), 2) AS realized_revenue,
303     ROUND(AVG(total_amount), 2) AS average_order_value
304 FROM orders
305 WHERE status != 'Cancelled';
```

Output Results

	valid_orders	realized_revenue	average_order_value
▶	8969	636476491.03	70964.04

Total Orders

8,969

Realized Revenue

\$636.48M

Average Order Value (AOV)

\$70,964.04

Category Performance: Volume vs Revenue Contribution

Aggregates sales by department to identify top revenue-generating product segments.

</> SQL Query

```
308 • SELECT
309     p.category,
310     SUM(oi.quantity) AS total_units_sold,
311     ROUND(SUM(oi.quantity * oi.item_price), 2) AS category_revenue,
312     ROUND(AVG(p.price), 2) AS avg_catalog_price
313 FROM order_items oi
314 INNER JOIN products p ON oi.product_id = p.product_id
315 GROUP BY p.category
316 ORDER BY category_revenue DESC;
```

Output Results

	category	total_units_sold	category_revenue	avg_catalog_price
▶	Electronics	6187	256279737.92	41908.05
	Furniture	7268	227395963.40	31052.50
	Automotive	7044	83854538.16	11890.14
	Sports	6185	44876608.38	7249.90
	Home & Kitch...	6382	42668979.92	6762.00
	Clothing	6744	16979648.33	2518.77
	Toys	6789	12539075.01	1856.93
	Beauty	6145	12384795.67	2018.29
	Books	7171	7149929.20	998.35
	Grocery	5784	4131299.63	716.58

Top Category Revenue (Electronics)

\$25.63M

Max Units Sold (Furniture)

7,268

Highest Avg Catalog Price

\$41,908.05

Payment Channel Adoption & Collection Breakdown

Analyzes payment method usage and transaction totals for financial reconciliation.

</> SQL Query

```
319 • SELECT
320     method,
321     COUNT(payment_id) AS transaction_count,
322     ROUND(SUM(amount_paid), 2) AS total_collected,
323     ROUND(AVG(amount_paid), 2) AS avg_transaction_value
324 FROM payments
325 GROUP BY method
326 ORDER BY total_collected DESC;
```

Output Results

	method	transaction_count	total_collected	avg_transaction_value
▶	Net Banking	1388	98829526.74	71202.83
	Credit Card	1388	97760269.88	70432.47
	Wallet	1366	97255355.86	71197.19
	UPI	1324	95232150.42	71927.61
	Debit Card	1325	94758080.64	71515.53
	COD	1311	93427590.49	71264.37

Top Channel Revenue (Net Banking)

98.83M

Total Transactions (All Channels)

8,102

Highest Avg Transaction (UPI)

71,927.61

Inventory Risk Assessment (Stockout & Zero-Sales Detection)

Identifies products currently out of stock alongside total sales volume to flag supply bottlenecks.

</> SQL Query

```
330 • SELECT
331     p.product_id,
332     p.name AS product_name,
333     p.category,
334     p.stock_quantity,
335     COALESCE(SUM(oi.quantity), 0) AS total_units_sold
336 FROM products p
337 LEFT JOIN order_items oi ON p.product_id = oi.product_id
338 WHERE p.stock_quantity = 0
339 GROUP BY p.product_id, p.name, p.category, p.stock_quantity
340 ORDER BY total_units_sold DESC;
```

Output Results

	product_id	product_name	category	stock_quantity	total_units_sold
▶	491	Premium Action Figure	Toys	0	154
	232	Advanced Cap	Clothing	0	137
	585	Deluxe Poetry Book	Books	0	128
	423	Smart Poetry Book	Books	0	126
	557	Portable Poetry Book	Books	0	122
	409	Deluxe Comic Collection	Books	0	121
	525	Classic Router	Electronics	0	119
	507	Everyday Green Tea Pack	Grocery	0	119
	474	Ultra Badminton Racket	Sports	0	116
	16	Eco LED TV	Electronics	0	114

Top Stockout Sales (Toys)

154

Total Stockout Items Listed

10

Most Impacted Segment (Books)

4 Items

Customer Lifetime Value (CLV) & High-Value Segment

Ranks top-spending customers using a multi-table join across profile and transaction data.

</> SQL Query

```
344 • SELECT
345     c.customer_id,
346     c.name AS customer_name,
347     COUNT(DISTINCT o.order_id) AS total_orders_placed,
348     ROUND(SUM(o.total_amount), 2) AS lifetime_spend
349 FROM customers c
350 INNER JOIN orders o ON c.customer_id = o.customer_id
351 WHERE o.status != 'Cancelled'
352 GROUP BY c.customer_id, c.name
353 ORDER BY lifetime_spend DESC
354 LIMIT 10;
```

Output Results

	customer_id	customer_name	total_orders_placed	lifetime_spend
▶	2800	Jatin Deshpande	9	1389561.83
	2308	Girindra Shetty	8	1342457.66
	622	Harish Devan	5	1286810.65
	1776	Nidra Chhabra	8	1201633.55
	831	Lakshit Rajagopal	9	1156354.75
	2621	Eshana Bajaj	7	1110892.17
	1756	Anika Dhaliwal	4	1106843.64
	752	Faqid Oak	8	1035011.12
	2003	Damini Garg	6	1032410.53
	2904	Advaith Samra	6	1029259.07

Top Customer Spend (Jatin Deshpande)

1.39M

Max Orders Placed

9 Orders

Top 10 Minimum Threshold

1.03M

Outlier Detection: Orders Exceeding Customer's Own Average

Uses a window function partition to isolate exceptionally large order values per user.

</> SQL Query

```
357 • WITH CustomerSpendProfile AS (  
358     SELECT  
359         order_id,  
360         customer_id,  
361         total_amount,  
362         order_date,  
363         AVG(total_amount) OVER(PARTITION BY customer_id) AS avg_cust_spend  
364     FROM orders  
365     WHERE status != 'Cancelled'  
366 )  
367 SELECT  
368     order_id,  
369     customer_id,  
370     total_amount,  
371     ROUND(avg_cust_spend, 2) AS customer_avg_order,  
372     ROUND(total_amount - avg_cust_spend, 2) AS spend_above_average  
373 FROM CustomerSpendProfile  
374 WHERE total_amount > (avg_cust_spend * 1.5)  
375 ORDER BY spend_above_average DESC;
```

Output Results

	order_id	customer_id	total_amount	customer_avg_order	spend_above_average
▶	7979	1163	827032.00	170142.34	656889.66
	1637	694	733530.33	200315.04	533215.29
	6488	2828	677201.39	198991.58	478209.81
	4760	1772	630443.88	167005.38	463438.50
	6658	1048	549766.38	109296.62	440469.76
	6937	2994	604672.44	169066.05	435606.39
	6815	1562	640896.55	211725.26	429171.29
	4437	2429	613871.71	205812.27	408059.44
	3765	2650	548577.69	147113.93	401463.76

Max Spend Above Average

656.9K

Highest Customer Average

211.7K

Outlier Threshold Scale

1.5x

Top-1 Highest Value Transaction Per Customer

Extracts full order details for each user's maximum transaction using window ranking.

SQL Query

```
379 • WITH RankedCustomerOrders AS (  
380     SELECT  
381         o.order_id,  
382         o.customer_id,  
383         c.name AS customer_name,  
384         o.order_date,  
385         o.total_amount,  
386         ROW_NUMBER() OVER (  
387             PARTITION BY o.customer_id  
388             ORDER BY o.total_amount DESC, o.order_id ASC  
389         ) AS ranking  
390     FROM orders o  
391     INNER JOIN customers c ON o.customer_id = c.customer_id  
392     WHERE o.status != 'Cancelled'  
393 )  
394 SELECT  
395     customer_id,  
396     customer_name,  
397     order_id AS highest_order_id,  
398     order_date,  
399     total_amount AS peak_spend_amount  
400 FROM RankedCustomerOrders  
401 WHERE ranking = 1  
402 ORDER BY peak_spend_amount DESC;
```

Output Results

	customer_id	customer_name	highest_order_id	order_date	peak_spend_amount
▶	1163	Zayyan Som	7979	2025-10-14 13:59:42	827032.00
	694	Charita Dhawan	1637	2026-08-12 04:27:06	733530.33
	2828	Gunbir Deep	6488	2024-11-07 06:04:18	677201.39
	723	Nitara Devan	3083	2026-08-06 02:10:55	656947.17
	1562	Harrison Kashyap	6815	2026-04-09 14:10:49	640896.55
	1772	Dhruv Soman	4760	2024-10-08 13:00:17	630443.88
	2429	Jason Palla	4437	2025-03-29 07:35:19	613871.71

Max Peak Transaction (Zayyan Som)

827.0K

Average Peak Transaction

678.9K

Transaction Cut-off

613.9K

Active vs. Disengaged User Base Analysis

Computes customer conversion efficiency by comparing ordering customers against inactive accounts.

</> SQL Query

```
406 • SELECT
407     COUNT(DISTINCT c.customer_id) AS total_registered_customers,
408     COUNT(DISTINCT o.customer_id) AS active_ordering_customers,
409     COUNT(DISTINCT c.customer_id) - COUNT(DISTINCT o.customer_id) AS inactive_customers,
410     ROUND((COUNT(DISTINCT o.customer_id) * 100.0) / COUNT(DISTINCT c.customer_id), 2) AS conversion_rate_pct
411 FROM customers c
412 LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

Output Results

	total_registered_customers	active_ordering_customers	inactive_customers	conversion_rate_pct
▶	3000	2902	98	96.73

Conversion Rate

96.73%

Active Ordering Customers

2,902

Inactive Customers

98

Key Insights

Synthesized business intelligence from customer conversion, spend variance, and high-value transactions.

 Search insights...



Overall Performance

\$636.48
M

Realized Revenue

Generated across 8,969 total orders.

Net Banking emerged as the top payment channel by revenue.



Category Leader

\$25.63M

Electronics Sales

Electronics category led revenue generation.

Primary category driver for the business.



Segment Volume

7,268

Furniture Units

Furniture segment recorded the highest volume sold.

Key category for high-volume sales and movement.



Customer Conversion

96.73%

Conversion Rate

Reached peak customer conversion.

Supported by 2,902 active ordering customers.

Conclusion

Key takeaways and strategic insights based on movie streaming database analytics.

Project Conclusion

This project successfully demonstrated the design and analysis of an Retail Store database using MySQL. Through table creation, data insertion, joins, aggregate functions, subqueries, and window functions, valuable insights were generated from the data.

The project strengthened my practical understanding of SQL and database analytics.